

Practical – 4

wait() with multiple child processes

```
practical4.c
/mnt/c/WINDOWS/system32/os/practical

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main()
7 {
8     int i;
9     pid_t pid;
10
11     printf("Parent process started. PID = %d\n", getpid());
12
13     // Create 3 child processes
14     for (i = 1; i <= 3; i++)
15     {
16         pid = fork();
17
18         if (pid < 0)
19         {
20             perror("fork failed");
21             exit(1);
22         }
23
24         if (pid == 0)
25         {
26             printf("Child %d: PID = %d, PPID = %d\n",
27                   i, getpid(), getppid());
28
29             sleep(1); // Different execution times
30
31             printf("Child %d finished. PID = %d\n",
32                   i, getpid());
33
34             exit(i);
35         }
36     }
37
38     // Parent waits for all children using wait()
39     for (i = 0; i < 3; i++)
40     {
41         int status;
42         pid_t child_pid;
43
44         child_pid = wait(&status);
45
46         if (WIFEXITED(status))
47         {
48             printf("Parent: Child PID %d completed with exit status %d\n",
49                   child_pid, WEXITSTATUS(status));
50         }
51     }
52
53     printf("Parent process finished.\n");
54
55     return 0;
56 }
```

```
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ gedit practical4.c

** (gedit:4647): WARNING **: 03:23:29.838: Could not load Peas repository: Typelib file f
or namespace 'Peas', version '1.0' not found

** (gedit:4647): WARNING **: 03:23:29.838: Could not load PeasGtk repository: Typelib fil
e for namespace 'PeasGtk', version '1.0' not found
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ gcc practical4.c -o practical4
gcc: fatal error: practical4.c: No such file or directory
compilation terminated.
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ gcc practical4.c -o practical4
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ ./practical4
Parent process started. PID = 6202
Child 1: PID = 6203, PPID = 6202
Child 2: PID = 6204, PPID = 6202
Child 3: PID = 6205, PPID = 6202
Child 1 finished. PID = 6203
Parent: Child PID 6203 completed with exit status 1
Child 2 finished. PID = 6204
Parent: Child PID 6204 completed with exit status 2
Child 3 finished. PID = 6205
Parent: Child PID 6205 completed with exit status 3
Parent process finished.
```

waitpid() with multiple children

```
Open  ▾  +  *waitpid_demo.c
/mnt/c/WINDOWS/system32/os/practical

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main()
7 {
8     pid_t children[3];
9     int i;
10
11     printf("Parent process started. PID = %d\n", getpid());
12
13     // Create 3 children
14     for (i = 0; i < 3; i++)
15     {
16         children[i] = fork();
17
18         if (children[i] < 0)
19         {
20             perror("fork failed");
21             exit(1);
22         }
23
24         if (children[i] == 0)
25         {
26             printf("Child %d: PID = %d, PPID = %d\n",
27                   i + 1, getpid(), getppid());
28
29             sleep(3 - i);
30
31             printf("Child %d finished. PID = %d\n",
32                   i + 1, getpid());
33
34             exit(i + 1);
35         }
36     }
37
38     // Parent waits for a specific child using waitpid()
39     for (i = 0; i < 3; i++)
40     {
41         int status;
42
43         waitpid(children[i], &status, 0);
44
45         if (WIFEXITED(status))
46         {
47             printf("Parent: Child PID %d completed with exit status %d\n",
48                   children[i], WEXITSTATUS(status));
49         }
50     }
51
52     printf("Parent process finished.\n");
53
54     return 0;
55 }
```

```
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ gcc waitpid_demo.c -o waitpid_demo
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ ./waitpid_demo
Parent process started. PID = 11938
Child 2: PID = 11940, PPID = 11938
Child 1: PID = 11939, PPID = 11938
Child 3: PID = 11941, PPID = 11938
Child 3 finished. PID = 11941
Child 2 finished. PID = 11940
Child 1 finished. PID = 11939
Parent: Child PID 11939 completed with exit status 1
Parent: Child PID 11940 completed with exit status 2
Parent: Child PID 11941 completed with exit status 3
Parent process finished.
```

Create and find Zombie Process

```
Open  ▾  +  *zombie.c
/mnt/c/WINDOWS/system32/os/practical

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4
5 int main()
6 {
7     pid_t pid;
8
9     pid = fork();
10
11     if (pid < 0)
12     {
13         perror("fork failed");
14         exit(1);
15     }
16
17     if (pid == 0)
18     {
19         printf("Child process running. PID = %d\n", getpid());
20         printf("Child process exiting...\n");
21
22         exit(0);
23     }
24     else
25     {
26         printf("Parent process running. PID = %d\n", getpid());
27         printf("Child PID = %d\n", pid);
28
29         // Parent does NOT call wait()
30         printf("Parent sleeping for 30 seconds...\n");
31
32         sleep(30);
33
34         printf("Parent exiting...\n");
35     }
36
37     return 0;
38 }
```

```
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ gcc zombie.c -o zombie
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ ./zombie
Parent process running. PID = 14770
Child PID = 14771
Parent sleeping for 70 seconds...
Child process running. PID = 14771
Child process exiting...
Parent exiting...
```

```
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ ps -ef | grep zombie
sampath    14770    1811    0 03:34 pts/0    00:00:00 ./zombie
sampath    14771    14770    0 03:34 pts/0    00:00:00 [zombie] <defunct>
sampath    14778    14705    0 03:34 pts/2    00:00:00 grep --color=auto zombie
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$
```

Eliminate the Zombie using wait() and waitpid()

```
Open  ▾  [icon]  zombie.c
/mnt/c/WINDOWS/system32/os/practical

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5
6 int main()
7 {
8     pid_t pid;
9     int status;
10
11     pid = fork();
12
13     if (pid < 0)
14     {
15         perror("fork failed");
16         exit(1);
17     }
18
19     if (pid == 0)
20     {
21         printf("Child process running. PID = %d\n", getpid());
22         sleep(2);
23         printf("Child process exiting...\n");
24         exit(0);
25     }
26     else
27     {
28         printf("Parent process running. PID = %d\n", getpid());
29         printf("Child PID = %d\n", pid);
30
31         // Parent waits for child
32         wait(&status);
33
34         if (WIFEXITED(status))
35         {
36             printf("Child completed successfully.\n");
37             printf("Child exit status = %d\n",
38                   WEXITSTATUS(status));
39         }
40         printf("Parent exiting...\n");
41     }
42
43     return 0;
44 }
```

```
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ ./zombie
```

```
Parent process running. PID = 16464
Child PID = 16465
Child process running. PID = 16465
Child process exiting...
Child completed successfully.
Child exit status = 0
Parent exiting...
```

```
sampath@MS-Desktop:/mnt/c/WINDOWS/system32/os/practical$ ps -ef | grep zombie
```

```
sampath    16464    1811    0 03:38 pts/0    00:00:00 ./zombie
sampath    16465    16464    0 03:38 pts/0    00:00:00 ./zombie
sampath    16472    14705    0 03:38 pts/2    00:00:00 grep --color=auto zombie
```